

Question 1:

A **Node** is one element of a linked list. It usually stores two things: the **data** (the value) and a link to the next node.

The **Head** is the first node of the linked list. It is the starting point used to reach the rest of the list.

A **Pointer** is the link/reference that tells a node where the next node is. In a singly linked list, this pointer only moves forward to the next node.

For 3 -> 16 -> 9 -> 21, the linked list would look like this:

head

|

v

[3 | *] -> [16 | *] -> [9 | *] -> [21 | None]

Here:

- head points to the first node
- each * means “points to the next node”
- the last node points to None, which means the chain ends

Question 2:

class Node:

def __init__(self, data):

self.data = data

self.next = None

def insert_at_end(head, data):

"""

Insert a new node with 'data' at the end of a singly linked list.

Returns the head of the list.

"""

new_node = Node(data)

Case 1: empty list

if head is None:

return new_node

Case 2: non-empty list, traverse to the last node

current = head

while current.next is not None:

current = current.next

current.next = new_node

return head

Example use:

Create list: 10 -> 20 -> 30

head = Node(10)

head.next = Node(20)

head.next.next = Node(30)

Insert 40 at the end

head = insert_at_end(head, 40)

Question 3:

A **Singly Linked List** node stores:

- the data
- a pointer to the **next** node only

So it can move only forward through the list.

A **Doubly Linked List** node stores:

- the data
- a pointer to the **next** node

- a pointer to the **previous** node

So it can move both forward and backward.

What each can/cannot do

- Singly linked list: cannot go backward directly
- Doubly linked list: can go forward and backward
- Singly linked list: deletion often needs the previous node
- Doubly linked list: deletion is easier because the node already knows both neighbors

Why deletion is easier in a Doubly Linked List

If we already have a reference to the node, we can directly update:

- `node.prev.next` to skip the node
- `node.next.prev` to skip the node

So we do not need to search for the previous node first, which saves time.

Question 4:

Operation	Array	Linked List
Access by index	$O(1)$	$O(n)$
Insert at beginning	$O(n)$	$O(1)$
Insert at end	$O(1)$	$O(n)$

Brief justification

Array access by index is $O(1)$ because arrays store elements in continuous memory, so the position can be calculated directly.

Linked list access by index is $O(n)$ because you must start from the head and move node by node until you reach the required position.

Question 5:

Here is a size(self) method for a linked list:

```
class Node:
```

```
    def __init__(self, data):
```

```
        self.data = data
```

```
        self.next = None
```

```
class LinkedList:
```

```
    def __init__(self):
```

```
        self.head = None
```

```
    def insert_at_end(self, data):
```

```
        new_node = Node(data)
```

```
        if self.head is None:
```

```
            self.head = new_node
```

```
        return
```

```
        current = self.head
```

```
        while current.next is not None:
```

```
            current = current.next
```

```
        current.next = new_node
```

```
    def size(self):
```

```
        """
```

```
        Returns the number of nodes in the linked list  
by traversing the list.
```

```
        """
```

count = 0

current = self.head

while current is not None:

count += 1

current = current.next

return count

Example:

ll = LinkedList()

ll.insert_at_end(5)

ll.insert_at_end(10)

ll.insert_at_end(15)

ll.insert_at_end(20)

print("Size of linked list:", ll.size())

Output:

Size of linked list: 4